

WEEK -5

Session-9:

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
root@DESKTOP-QU0HVJI: ~/! × + v
GNU nano 7.2 producer_consumer.c *
}

if (chdir(path) == -1)
{
    perror("cd");
    return;
}

if (getcwd(newpwd, sizeof(newpwd)) == NULL)
{
    perror("getcwd");
    return;
}

setenv("OLDPWD", oldpwd, 1);
setenv("PWD", newpwd, 1);
}

int main()
{
    char command[100];

    while (1)
    {
        printf("shell> ");
        fflush(stdout);

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            break;
        }

        else if (strcmp(command, "cd") == 0)
        {
            change_directory(NULL);
        }

        else if (strncmp(command, "cd ", 3) == 0)
        {
            change_directory(command + 3);
        }

        else
        {
            printf("Unknown command\n");
        }
    }

    return 0;
}
```

OUTPUT:

```
root@DESKTOP-QU0HVJI:~/SKILL/skill-5# gcc producer_consumer.c -o producer_consumer
root@DESKTOP-QU0HVJI:~/SKILL/skill-5# ./producer_consumer
shell> pwd
Unknown command
shell> cd /tmp
shell> cd -
/root/SKILL/skill-5
shell> exit
root@DESKTOP-QU0HVJI:~/SKILL/skill-5# |
```

Report: The cd command was implemented to change the current working directory using chdir().

The program validates the given path and displays an error for invalid paths.

PWD and OLDPWD are maintained to track the current and previous directories.

Question 2: TO Identify Built-ins, Create Dispatch Table, Execute In- Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

root@DESKTOP-QU0HWJL: ~/! × + v

GNU nano 7.2

dispatch_shell.c *

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

void pwd_command(char *arg)
{
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
    else
        perror("pwd");
}

void cd_command(char *arg)
{
    if (arg == NULL || strcmp(arg, "~") == 0)
    {
        arg = getenv("HOME");
    }

    if (chdir(arg) == -1)
        perror("cd");
}

void exit_command(char *arg)
{
    exit(0);
}

struct builtin
{
    char *name;
    void (*function)(char *);
};

struct builtin builtins[] =
{
    {"pwd", pwd_command},
    {"cd", cd_command},
    {"exit", exit_command}
};

int main()
{
    char input[200];

    while (1)
    {
        printf("shell> ");
        fflush(stdout);

        fgets(input, sizeof(input), stdin);

        input[strcspn(input, "\n")] = '\0';

        char *command = strtok(input, " ");
        char *arg = strtok(NULL, "");

        if (command == NULL)
            continue;
    }
}
```

OUTPUT:

```
root@DESKTOP-QU0HVJI:~/SKILL/skill-5# nano dispatch_shell.c
root@DESKTOP-QU0HVJI:~/SKILL/skill-5# gcc dispatch_shell.c -o dispatch_shell
root@DESKTOP-QU0HVJI:~/SKILL/skill-5# ./dispatch_shell
shell> pwd
/root/SKILL/skill-5
shell> cd/ tmp
Command: No such file or directory
shell> pwd
/root/SKILL/skill-5
shell> /tmp
Command: Permission denied
shell> export NAME=HITESH
Command: No such file or directory
shell> ls
dispatch_shell dispatch_shell.c producer_consumer producer_consumer.c
shell> exit
root@DESKTOP-QU0HVJI:~/SKILL/skill-5# |
```

Report: A dispatch table was created to identify and execute built-in commands such as cd, pwd, export, and exit.

Function pointers were used to connect commands with their respective functions.

External commands were handled using fork() and exec().

Session : 10

Question 3: To Retrieve Current Directory, Display path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

```
root@DESKTOP-QU0HVJI: ~/s × + v
GNU nano 7.2 pwd_exit_shell.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

void pwd_command()
{
    char path[1024];

    if (getcwd(path, sizeof(path)) != NULL)
        printf("%s\n", path);
    else
        perror("pwd");
}

void exit_command()
{
    printf("Exiting shell...\n");
    printf("Resources cleaned successfully.\n");
    exit(0);
}

int main()
{
    char input[100];

    while (1)
    {
        printf("shell> ");

        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "pwd") == 0)
        {
            pwd_command();
        }
        else if (strcmp(input, "exit") == 0)
        {
            exit_command();
        }
        else
        {
            printf("Unknown command\n");
        }
    }

    return 0;
}
```

OUTPUT:

```
root@DESKTOP-QU0HVJI:~/SKILL/skill-5# nano pwd_exit_shell.c
root@DESKTOP-QU0HVJI:~/SKILL/skill-5# gcc pwd_exit_shell.c -o pwd_exit_shell
root@DESKTOP-QU0HVJI:~/SKILL/skill-5# ./pwd_exit_shell
shell> pwd
/root/SKILL/skill-5
shell> hello
Unknown command
shell> exit
Exiting shell...
Resources cleaned successfully.
root@DESKTOP-QU0HVJI:~/SKILL/skill-5# |
```

Report: The pwd command was implemented using getcwd() to display the current working directory.

The exit command terminates the shell after performing the required cleanup.

The program was tested to verify correct directory display and termination.

Question 4: To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

```
root@DESKTOP-QU0HVJI: ~/? × + ∨
GNU nano 7.2 export.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

int valid_name(char *name)
{
    if (!name || !*name)
        return 0;

    if (!isalpha(*name) && *name != '_')
        return 0;

    for (char *p = name + 1; *p; p++)
    {
        if (!isalnum(*p) && *p != '_')
            return 0;
    }

    return 1;
}

void export_command(char *input)
{
    char *equal = strchr(input, '=');

    if (equal == NULL)
    {
        printf("export: invalid format\n");
        return;
    }

    *equal = '\0';

    char *name = input;
    char *value = equal + 1;

    if (!valid_name(name))
    {
        printf("export: invalid variable name\n");
        return;
    }

    if (setenv(name, value, 1) == -1)
    {
        perror("export");
        return;
    }

    printf("Exported %s=%s\n", name, value);
}

int main()
{
    char input[200];

    while (1)
    {
        printf("shell> ");

        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0';

        if (strncmp(input, "export ", 7) == 0)
```

OUTPUT:

```
Resources cleaned successfully.  
root@DESKTOP-QU0HVJI:~/SKILL/skill-5# nano export.c  
root@DESKTOP-QU0HVJI:~/SKILL/skill-5# gcc export.c -o export  
root@DESKTOP-QU0HVJI:~/SKILL/skill-5# ./export  
shell> export NAME=Hitesh  
Exported NAME=Hitesh  
shell> print NAME  
Hitesh  
shell> LL  
Unknown command  
shell> |
```

Report:

The export command was implemented to create and update environment variables using `setenv()`.

Variable names are validated before storing their values.

The variables were tested using `getenv()` to verify that they were correctly stored and accessed.

